自动舵调试界面 使用说明

> **文件**：`control/ui/debug_ui.html` **类型**：单文件 Web 应用（HTML + CSS + JS，无第三方依赖） **运行方式**：浏览器直接打开即可

---

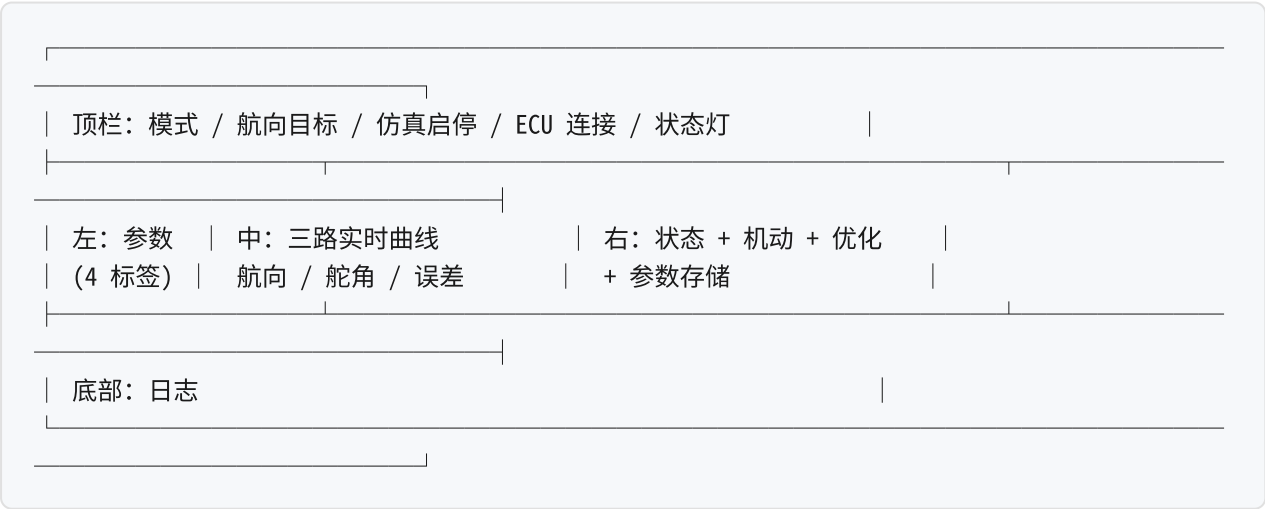
1. 概述

调试界面根据 `control/include/` 下的 C++ 控制算法参数编写，提供：

- **参数在线编辑**：PD、增益调度、辨识器、自动优化全部参数
- **实时曲线**：航向、舵角、误差/转向率三路 Canvas 绘图
- **状态显示**：航向/舵角/误差/船速/海况/模式/$\hat{K}$/$\hat{T}$/J/ζ/ω_n
- **机动触发**：Williamson / U-Turn / Zigzag / Circles / Cloverleaf / Search 一键启动
- **自动优化控制**：启用/禁用/ESC 一步/回退基线
- **参数存储**：保存/加载，含版本号与 CRC
- **双模式运行**：内置仿真器（独立运行）/ WebSocket 对接真实 ECU

---

2. 界面布局



左侧参数面板（4 个标签）

标签	对应 C++ 模块	参数
PD	`pd_controller.hpp`	Kp, Kd, dMax, ddMax, deadzone, tau, quant, dt
调度	`gain_schedule.hpp`	调度表（船速/海况 → Kp/Kd），可增删行
辨识	`nomoto_identifier.hpp`	λ, dt, δ激励, r激励, P0
优化	`auto_tuner.hpp`	ζ, ωn, ESC 参数, 性能权重, 安全约束

中部曲线区

曲线	内容	量程
航向	蓝=实际航向 / 红=目标航向	±180°
舵角	绿=指令舵角 / 黄=实际舵角	±35°
误差/率	紫=航向误差 / 青=转向率	±20

右侧状态与控制

- **实时状态**：8 项关键指标
- **机动触发**：7 个预置机动 + 停止
- **自动优化**：$\hat{K}$/$\hat{T}$/J/$\hat{\zeta}$/$\hat{\omega}$n/激励 + 启用/禁用/ESC/回退
- **参数存储**：版本/CRC/时间戳 + 保存/加载

---

3. 快速开始

3.1 独立运行（内置仿真器）

1. 浏览器打开 `debug_ui.html`
2. 顶部点击 **"启动"** 按钮，仿真器以 50 Hz 运行
3. 在 **"航向目标°"** 输入 90，点击 **"应用"**
4. 观察航向曲线跟踪至 90°
5. 调整左侧 PD 参数，点击 **"应用 PD 参数"** 观察响应变化

3.2 对接真实 ECU（WebSocket）

1. ECU 端实现 WebSocket 服务端（默认 `ws://localhost:8080`）
2. 顶栏输入 ECU 地址，点击 **"连接 ECU"**
3. 连接成功后状态灯变绿
4. ECU 按 JSON 协议上报传感器数据，UI 下发指令

---

4. WebSocket 协议

4.1 UI → ECU（指令）

```
{
  "type": "cmd",
  "rudder": 3.2,
  "mode": "AUTO_HEADING",
  "headingRef": 128.5
}
```

字段	类型	说明
type	string	"cmd"
rudder	double	目标舵角 (deg)
mode	string	MANUAL / AUTO_HEADING / AUTO_MANEUVER / HOLD
headingRef	double	目标航向 (deg)

4.2 ECU → UI（传感器上报）

```
{
  "type": "sensor",
  "sensor": {
    "headingDeg": 128.4,
    "rateDegS": 0.5,
    "rudderDeg": 3.2,
    "speedKn": 12.3,
    "seaState": 1
  }
}
```

4.3 参数下发（UI → ECU）

应用参数时（PD/调度/辨识/优化/存储/模式）会自动通过 WebSocket 下发，消息格式：

```
{
  "type": "set_params",
  "scope": "pd",
  "ts": 1722146400,
  "pd": { "Kp": 1.5, "Kd": 0.6, "dMax": 35, "ddMax": 10,
          "deadzone": 0.3, "tau": 0.2, "quant": 0.1, "dt": 0.02 }
}
```

scope	字段	说明
`pd`	pd	PD 控制器全部参数
`sched`	sched	增益调度表（数组）
`id`	id	辨识器参数
`tuner`	tuner	自动优化参数
`store`	store	参数存储集（含 version/crc）
`mode`	mode, headingRef	模式与航向目标

实时滑块拖动 Kp/Kd/航向目标时也会触发下发（仅当 ECU 已连接）。

4.4 ECU → UI（参数确认）

ECU 收到参数后回复确认：

```
{ "type": "params_ack", "scope": "pd", "version": 13, "accepted": true }
```

4.5 ECU → UI（故障上报）

```
{ "type": "status", "faults": ["E001"] }
```

---

5. 与 C++ 代码的参数映射

UI 参数	C++ 字段	默认值
Kp	`PdController::Params::Kp`	1.5
Kd	`PdController::Params::Kd`	0.6
舵角限幅	`PdController::Params::dMax`	35
舵速限幅	`PdController::Params::ddMax`	10
死区	`PdController::Params::deadzone`	0.3
滤波τ	`PdController::Params::tau`	0.2
量化	`PdController::Params::quant`	0.1
周期	`PdController::Params::dt`	0.02
遗忘因子λ	`NomotoIdentifier::Params::lambda`	0.98
δ激励	`NomotoIdentifier::Params::deltaMin`	3
r激励	`NomotoIdentifier::Params::rMin`	0.5
P0	`NomotoIdentifier::Params::p0`	1e6
ζ目标	`AutoTuner::Params::zeta`	0.85
ωn	`AutoTuner::Params::wn`	0.20
ESC 扰动幅	`AutoTuner::Params::escAmp`	0.05
ESC 频率	`AutoTuner::Params::escFreq`	0.05
ESC 步长	`AutoTuner::Params::escStep`	0.02
wU	`AutoTuner::Params::wU`	0.01
wDu	`AutoTuner::Params::wDu`	0.005
更新幅≤	`AutoTuner::Params::maxUpdateFrac`	0.10
ζmin	`AutoTuner::Params::zetaMin`	0.6
ωn max	`AutoTuner::Params::wnMax`	0.5

UI 参数	C++ 字段	默认值
振荡°	`AutoTuner::Params::oscMax`	5
高速去激活	`AutoTuner::Params::highSpeedDeactivateKn`	15
窗口	`AutoTuner::Params::windowSec`	60

---

6. 内置仿真器

UI 内置一个 **Nomoto 一阶船舶模型** 仿真器，用于无 ECU 时的独立调试：

$$T\dot{r} + r = K\delta$$

- 默认参数：$K = 0.18$，$T = 8.5$
- 舵角一阶滞后跟随（时间常数 0.5 s）
- 注入小幅噪声模拟传感器抖动
- 仿真周期 = PD 控制周期 `dt`

可修改 `State.sim.K`、`State.sim.T` 调整船舶动态。

---

7. 使用场景示例

7.1 PD 参数整定

1. 启动仿真器
2. 设置航向目标 90°，观察响应
3. 增大 Kp 加快响应，增大 Kd 抑制超调
4. 调整死区/滤波τ 观察抖动
5. 满意后点击 **"保存"** 固化参数

7.2 实时滑块调节

左侧顶部 5 个滑块支持拖动实时调节：

滑块	作用	范围
Kp	比例增益	0~5
Kd	微分增益	0~2
航向目标	目标航向	-180~180°
船速	仿真船速	0~25 kn
海况	仿真海况	0~3

- Kp/Kd 滑块拖动时同步更新 PD 参数输入框，ECU 连接时自动下发
- 航向目标滑块拖动时实时改变目标航向
- 船速/海况滑块用于仿真器工况切换，观察增益调度自动切换

7.3 自动优化验证

1. 切到"优化"标签，设置 ζ/ωn 目标
2. 点击 **"启用"** 开启自动优化
3. 观察右侧 $\hat{K}/\hat{T}$ 收敛、J 下降、$\hat{\zeta}/\hat{\omega}_n$ 趋近目标
4. 高速（>15 kn）时自动去激活

7.4 机动测试

1. 启动仿真器，航向稳定
2. 点击 **"Williamson"** 触发威廉逊掉头
3. 观察"当前段"推进：1/3 → 2/3 → 3/3
4. 完成后自动切回 AUTO_HEADING

7.5 增益调度编辑

1. 切到"调度"标签
2. 修改船速/海况/Kp/Kd，或点击 **"+ 新增行"**
3. 点击 **"应用调度表"**
4. 改变船速（修改 `State.sensor.speedKn`），观察自动切换增益

---

8. 数据导出

顶栏"数据"组提供三种操作：

按钮	格式	内容
导出 CSV	CSV	原始时间序列：index, heading, headingRef, rudder, rudderCmd, error, rate
导出 JSON	JSON	含元信息 + 参数快照 + 全部历史数据
清空	—	清空历史缓冲区

导出文件名带时间戳，如 `autopilot_2026-07-29T13-20-00.csv`。

CSV 示例：

```
index,heading,headingRef,rudder,rudderCmd,error,rate
0,0.000,0.000,0.000,0.000,0.000,0.000
1,0.012,0.000,0.000,0.020,-0.012,0.005
...
```

JSON 结构：

```
{
  "meta": { "exportedAt": "...", "sampleCount": 600, "dt": 0.02, "mode": "AUTO_HEADING" },
  "params": { "pd": {...}, "id": {...}, "tuner": {...} },
  "data": { "heading": [...], "headingRef": [...], "rudder": [...],
            "rudderCmd": [...], "error": [...], "rate": [...] }
}
```

导出数据可用于：

- 岸基离线分析（MATLAB/Python）
- 算法参数回归验证
- 故障事后回溯

---

9. 注意事项

- **纯算法层**：UI 中的算法实现与 C++ 头文件一一对应，仅用于调试可视化，**不可直接用于实船控制**。实船必须使用 C++ ECU。

- **WebSocket 可选**：未连接 ECU 时使用内置仿真器；连接后仿真器自动旁路，UI 仅显示 ECU 上报数据。
- **参数持久化**：UI 的"保存/加载"仅在内存中；真实持久化由 ECU 端 `ParamStore` 完成。
- **浏览器兼容**：需支持 Canvas、WebSocket、ES6（Chrome/Firefox/Edge 最新版均可）。

---

10. 文件清单

```
control/ui/
├──── debug_ui.html       # 调试界面（单文件，933 行）
└──── README.md           # 本说明文档
```

— 文档结束 —